

INFO0009: WeND(Y)'s Party Management System

Aleksandr PAVLOV, Marco KUCKART, Adrien CORMANN

1 Architecture du site web

Chaque page du site est composée d'un menu de navigation à gauche, qui permet de sélectionner une des 9 pages de notre site, ainsi que du contenu même de la page. Ces pages sont :

- Tables :
 - Clients : permet d'afficher et de sélectionner des clients.
 - Locations : permet d'afficher et de sélectionner des lieux.
- Events :
 - Event Dashboard : le tableau de bord des événements.
 - Events : permet d'afficher et de sélectionner des événements.
 - New Event : permet d'ajouter des événements dans la base de donnée.
- CD :
 - Cd Dashboard : affiche la disponibilité des CDs.
 - Cd Stats : le tableau de bord des CDs, permet aussi de modifier les CDs.
 - Cd New : permet d'ajouter des CDs dans la base de donnée.
- Genres :
 - Taxonomy : permet d'ajouter des informations sur la taxonomie des genres.

2 Opérations d'initialisation

Au démarrage du conteneur Docker, la base de données est initialisée automatiquement. Le script est divisé en huit fichiers qui définissent la structure. La répartition s'est faite selon les catégories suivantes :

- Création et définition des tables SQL.
- Procédures de modification des éléments de la base.
- Fonctions de calcul et de traitement.
- Définition des vues.
- Fonctions spécifiques aux vues.
- Importation des données à partir des fichiers .csv.
- Mise en place des triggers.

L'exécution de ce script au démarrage garantit qu'aucune structure n'est utilisée avant son initialisation.

3 Description des vues, requêtes, transactions complexes

Dans cette section, nous n'expliquons pas les requêtes basiques.

3.1 Vues

Nous utilisons des vues dans la base de donnée, afin de simplifier l'accès aux données complexes. Pour la taxonomie des genres, nous nous basons sur une vue récursive qui nous permet pour chaque genre, d'avoir l'ensemble des sous genre de celui-ci.

```

1 CREATE VIEW specializes_transitive_closure AS
2   WITH RECURSIVE specializes_transitive(genre, subgenre) AS (
3     SELECT genre, subgenre FROM specializes
4     UNION ALL
5     SELECT st.genre, s.subgenre
6     FROM specializes_transitive st, specializes s
7     WHERE st.subgenre = s.genre
8   )
9   SELECT * FROM specializes_transitive;

```

specializes_transitive_closure

Pour les statistiques sur les CDs, nous calculons d'abord des statistiques avec des fonctions utilitaires simples, puis pour les genres, nous faisons l'union entre les genres présents sur les chansons du CD et tous les sous-genres de ces genres là, en utilisant la vue précédente.

```

1 DROP VIEW IF EXISTS cd_statistics;
2 CREATE VIEW cd_statistics AS
3   SELECT
4     cd.cd_number,
5     cd_get_total_song_duration(cd.cd_number) AS total_duration,
6     cd_get_min_song_duration(cd.cd_number) AS min_duration,
7     cd_get_max_song_duration(cd.cd_number) AS max_duration,
8     cd_get_average_song_duration(cd.cd_number) AS average_duration,
9     cd_playlist_count(cd.cd_number) AS playlist_count,
10    (
11      SELECT GROUP_CONCAT(all_genres.genre ORDER BY all_genres.genre
12        SEPARATOR ',')
13      FROM (
14        SELECT song.genre
15        FROM song
16        WHERE song.cd_number = cd.cd_number
17        UNION
18        SELECT stc.genre
19        FROM song
20        JOIN specializes_transitive_closure stc ON stc.subgenre = song.
21        genre
22        WHERE song.cd_number = cd.cd_number
23      ) AS all_genres
24    ) AS genres
25  FROM cd;

```

cd_statistics

Pour le dashboard des CDs, nous commençons par générer toutes les dates entre le tout premier et le tout dernier événement. Ensuite pour chaque élément du produit cartésien de ces dates et des CD, on calcule le nombre de fois que ce CD est utilisé ce jour là. Notons que si il y a un grand écart entre les deux dates et que la base de donnée contient beaucoup de CDS, le produit cartésien générera une très grande quantité de tuples.

```

1 CREATE VIEW cd_dashboard AS
2   WITH RECURSIVE dates(date) AS (
3     SELECT MIN(date) AS date FROM event
4     UNION ALL
5     SELECT DATE_ADD(date, INTERVAL 1 DAY)
6     FROM dates
7     WHERE date < (SELECT MAX(date) FROM event)
8   )
9   SELECT
10     dates.date,
11     cd.cd_number,

```

```

12         cd.copies,
13         (
14             SELECT COUNT(DISTINCT event.id)
15             FROM event
16             JOIN contains ON event.playlist = contains.playlist
17             WHERE contains.cd_number = cd.cd_number AND event.date = dates.date
18         ) AS copies_used
19 FROM dates
20 CROSS JOIN cd;

```

cd_dashboard

3.2 Requêtes

Pour savoir si une playlist est disponible, nous commençons par vérifier qu'elle existe, et puis pour chaque CD, on compte le nombre de fois qu'il est utilisé dans un autre événement à la même date. Si ce nombre est supérieur au nombre de copies du CD, la playlist n'est pas disponible.

```

1 SELECT (
2     SELECT COUNT(*) FROM playlist WHERE name = :playlist_name
3 ) = 1
4 AND (
5     SELECT COUNT(*) = 0
6     FROM contains c JOIN cd USING (cd_number)
7     WHERE c.playlist = :playlist_name
8     AND (
9         SELECT COUNT(*) FROM event e
10        JOIN contains c2 ON c2.playlist = e.playlist
11        AND c2.cd_number = c.cd_number
12        WHERE e.date = :date_event AND e.id != :id_actuel
13    ) >= cd.copies
14 ) AS is_available;

```

Requête de vérification de disponibilité de playlist

3.3 Transactions

Toute modification apportée à la base de données passe par une transaction. Ce mécanisme garantit l'intégrité des données : en cas de succès, les changements sont validés définitivement (commit), en cas d'erreur, le système effectue un retour à l'état initial (rollback).

Afin d'assurer la cohérence lors d'opérations simultanées, nous utilisons des verrous (locks) sur les données concernées, empêchant ainsi tout conflit durant l'exécution des modifications.

4 Répartition du travail

1. Aleksandr Pavlov : Code du site web et des requêtes SQL pour accéder aux données, architecture globale du programme.
2. Marco Kuckart : Création des tables de la base de donnée, de certaines vues, et des triggers/procédure veillant à l'intégrité de la base de donnée.
3. Adrien Cormann : Coach sportif.